THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Computer Science

Bachelor of Science in Computer Science

Software Requirements Specification (SRS)

AI-Powered Exam Paper Generator System

Field	Details
Project Title	AI-Powered Exam Paper Generator
Document Type	Software Requirements Specification (SRS)
Version	1.0
Date	April 2025
Prepared By	Mujeeb ur Rehman (21886003) | Madad Abbas (21886010)
Semester	7th Semester, BSCS
Supervisor	Syeda Asma bibi
University	The Islamia University of Bahawalpur

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the AI-Powered Exam Paper Generator System. This document is intended for the development team, academic supervisor, and university evaluators as part of the Final Year Project (FYP) for the Bachelor of Science in Computer Science program at The Islamia University of Bahawalpur.

The system aims to automate the process of generating examination papers by leveraging Artificial Intelligence and Natural Language Processing technologies, reducing the time and effort required by educators to create high-quality, curriculum-aligned assessments.

1.2 Scope

The AI-Powered Exam Paper Generator is a web-based application that allows teachers and educators to:

- Upload course materials in PDF or image format
- Specify examination parameters (question types, count, difficulty level)
- Automatically generate structured exam papers using a locally-hosted Large Language Model (LLaMA 3 8B via Ollama)
- Download formatted exam papers in PDF and DOCX formats

The system will use OCR (Optical Character Recognition) for text extraction from images, a Retrieval-Augmented Generation (RAG) pipeline for intelligent content retrieval, and Bloom's Taxonomy for cognitive difficulty classification.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
API	Application Programming Interface
FastAPI	A modern, fast Python web framework for building APIs
Ollama	A tool for running large language models locally
LLaMA 3	Meta's open-source Large Language Model (8B parameters)
ChromaDB	An open-source vector database for embedding storage
Bloom's Taxonomy	A hierarchical model of cognitive learning objectives

Term	Definition
MCQ	Multiple Choice Question
DOCX	Microsoft Word Open XML Document format
FYP	Final Year Project
UI	User Interface

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Meta LLaMA 3 Technical Report, 2024
- Lewis et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- Bloom, B.S. (1956): Taxonomy of Educational Objectives
- FastAPI Official Documentation: https://fastapi.tiangolo.com
- Ollama Official Documentation: https://ollama.com

1.5 Overview

This SRS is organized as follows:

- Section 1 provides an introduction, scope, and definitions
- Section 2 describes the overall system description and constraints
- Section 3 details specific functional requirements
- Section 4 outlines non-functional requirements
- Section 5 presents system models and use cases
- Section 6 covers external interface requirements

2. Overall Description

2.1 Product Perspective

The AI-Powered Exam Paper Generator is a standalone web application. It is not a replacement for a full Learning Management System (LMS) but rather a specialized tool designed to assist educators in the time-consuming task of exam preparation. The system operates entirely on local infrastructure — no third-party AI API (e.g., OpenAI) is required, making it cost-effective and privacy-preserving.

The system interfaces with:

- The local file system (for uploaded documents and generated outputs)
- Ollama (local LLM server running LLaMA 3 8B)
- ChromaDB (local vector database for RAG)
- Browser-based frontend (React or plain HTML/JS)

2.2 Product Functions

The primary functions of the system are:

1. Document Upload & Processing — Accept PDF and image files, extract text using PyMuPDF and Tesseract OCR
2. Text Chunking & Embedding — Split extracted text into semantic chunks, embed using Sentence Transformers, store in ChromaDB
3. Question Generation — Use LLaMA 3 via Ollama to generate MCQs, short questions, and long questions based on retrieved context
4. Bloom's Taxonomy Classification — Categorize questions by cognitive level (Remember, Understand, Apply, Analyze, Evaluate, Create)
5. Question Deduplication — Detect and remove semantically duplicate questions using cosine similarity
6. Exam Paper Formatting — Assemble generated questions into a professional exam paper layout
7. Export — Download the exam paper as a formatted PDF or DOCX file
8. Question Bank — Persist generated questions in a local database for future reuse

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Teacher / Educator	Primary user who uploads course material and generates exam papers	Low to Medium
System Administrator	Manages the server environment, Ollama model, and deployments	High
Student (Future Scope)	May use the system for self-assessment practice paper generation	Low

2.4 Operating Environment

- Operating System: Windows 10 / Windows 11 (64-bit)
- Browser: Chrome 90+, Firefox 88+, Edge 90+
- RAM: Minimum 16 GB (for LLaMA 3 8B model)
- Storage: Minimum 20 GB free (model + database + uploads)
- Python: 3.10 or higher (installed via python.org installer)
- Node.js: 18+ (for frontend build tools, if React is used)
- Ollama: Latest Windows release with LLaMA 3 8B model pulled

2.5 Design and Implementation Constraints

- The LLM must run locally via Ollama — no cloud API dependencies
- The system must handle PDFs up to 50 MB and up to 10 image files per session
- OCR must support English and basic Urdu text
- The system must not store uploaded documents permanently after session ends (privacy)

- All generated output must be properly attributed as AI-assisted

2.6 Assumptions and Dependencies

- Ollama is pre-installed and the LLaMA 3:8B-instruct-q4_K_M model is available locally
- The host machine has a GPU or sufficient CPU for inference (GGUF quantized model)
- Users upload legible, readable documents (very low-quality scans may reduce OCR accuracy)
- Internet connection is not required for core functionality (fully offline-capable)

3. Specific Requirements

3.1 Functional Requirements

3.1.1 File Upload and Management

Req ID	Requirement	Priority
FR-01	The system shall allow users to upload PDF files up to 50 MB	High
FR-02	The system shall allow users to upload up to 10 image files (JPG, PNG) per session	High
FR-03	The system shall validate file types and reject unsupported formats	High
FR-04	The system shall display upload progress to the user	Medium
FR-05	The system shall automatically delete uploaded files after session ends or after 24 hours	Medium

3.1.2 Text Extraction and OCR

Req ID	Requirement	Priority
FR-06	The system shall extract native text from PDFs using PyMuPDF	High
FR-07	The system shall apply Tesseract OCR for scanned PDFs and image files	High
FR-08	The system shall preprocess images (grayscale, binarization, noise reduction) before OCR	High
FR-09	The system shall detect OCR confidence and warn the user if text quality is low	Medium
FR-10	The system shall clean extracted text by removing headers, footers, page numbers, and artifacts	High

3.1.3 Question Generation

Req ID	Requirement	Priority
FR-11	The system shall allow users to specify the number of MCQs (0-50)	High
FR-12	The system shall allow users to specify the number of short questions (0-20)	High
FR-13	The system shall allow users to specify the number of long/essay questions (0-10)	High
FR-14	The system shall allow users to select difficulty level: Easy, Medium, or Hard	High
FR-15	The system shall generate MCQs with exactly 4 options and one correct answer	High
FR-16	The system shall include an explanation for each MCQ answer	Medium
FR-17	The system shall map questions to Bloom's Taxonomy levels	Medium
FR-18	The system shall generate questions strictly based on uploaded content only	High
FR-19	The system shall detect and remove duplicate or near-duplicate questions	Medium

3.1.4 Exam Paper Export

Req ID	Requirement	Priority
FR-20	The system shall format generated questions into a structured exam paper layout	High
FR-21	The system shall allow download of exam paper in DOCX format	High
FR-22	The system shall allow download of exam paper in PDF format	High
FR-23	The system shall include university/institution name, subject, date, and time fields in the paper	Medium
FR-24	The system shall display a topic coverage summary after generation	Low

3.1.5 Question Bank

Req ID	Requirement	Priority
FR-25	The system shall store all generated questions in a persistent local database (SQLite)	Medium
FR-26	The system shall allow users to regenerate a paper by pulling from the question bank	Medium
FR-27	The system shall allow users to view, edit, and delete saved questions	Low

3.2 Non-Functional Requirements

3.2.1 Performance

- The system shall complete text extraction for a 20-page PDF within 30 seconds
- The system shall generate up to 20 questions within 3 minutes on a mid-range machine (Intel i5, 16GB RAM)
- The system shall support at least 5 concurrent users in a local network deployment

3.2.2 Reliability

- The system shall implement retry logic (up to 3 attempts) for failed LLM responses
- The system shall validate all LLM outputs before displaying to the user
- The system shall gracefully handle corrupt or unreadable files without crashing

3.2.3 Security

- Uploaded files shall not be accessible by other users in a shared deployment
- No user data shall be sent to external servers or APIs
- Session-based file isolation shall be enforced via unique session tokens

3.2.4 Usability

- The interface shall be usable by a non-technical educator with minimal training
- All error messages shall be presented in plain language
- The system shall provide real-time progress feedback during generation

3.2.5 Maintainability

- The codebase shall follow modular architecture with clear separation of concerns
- All modules shall include inline documentation and docstrings
- The system shall support swapping the LLM model (e.g., switching from LLaMA 3 to Mistral) via configuration

4. External Interface Requirements

4.1 User Interfaces

- Web-based frontend accessible via modern browser
- File upload area with drag-and-drop support
- Form controls for specifying MCQ count, short question count, long question count, and difficulty
- Real-time progress bar during generation
- Preview pane for generated exam paper before download
- Download buttons for PDF and DOCX

4.2 Hardware Interfaces

- Local machine file system (for uploads and outputs)
- GPU (optional): NVIDIA CUDA-compatible GPU for faster LLM inference

4.3 Software Interfaces

Component	Technology	Purpose
Backend Framework	FastAPI (Python)	REST API and request handling
LLM Engine	Ollama + LLaMA 3 8B (q4_K_M)	AI question generation
Vector Database	ChromaDB	RAG document embedding storage
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)	Text vectorization
PDF Text Extraction	PyMuPDF (fitz)	Native PDF text extraction
OCR Engine	Tesseract + pytesseract	Image and scanned PDF text extraction
Document Export	python-docx	DOCX generation
PDF Export	WeasyPrint / ReportLab	PDF generation
Database	SQLite	Question bank persistence
Frontend	React.js or plain HTML/CSS/JS	User Interface

5. Use Cases

5.1 Use Case Summary

Use Case ID	Use Case Name	Actor	Priority
UC-01	Upload Course Material	Teacher	High
UC-02	Configure Exam Parameters	Teacher	High
UC-03	Generate Exam Paper	System	High
UC-04	Preview Generated Paper	Teacher	Medium
UC-05	Download Exam Paper (PDF/DOCX)	Teacher	High
UC-06	View Topic Coverage Map	Teacher	Low
UC-07	Access Question Bank	Teacher	Medium
UC-08	Regenerate from Question Bank	Teacher	Medium

5.2 Detailed Use Case: UC-03 — Generate Exam Paper

Field	Description
Use Case ID	UC-03
Name	Generate Exam Paper
Actor	Teacher (primary), System (secondary)
Preconditions	User has uploaded at least one valid file and configured exam parameters
Main Flow	1. User clicks Generate button2. System validates inputs3. OCR pipeline extracts and cleans text4. Text is chunked and embedded into ChromaDB5. System retrieves relevant chunks via RAG6. LLaMA 3 generates questions per type and difficulty7. Questions are validated and deduplicated8. Bloom's Taxonomy labels are assigned9. Exam paper is assembled and displayed in preview
Alternate Flow	If LLM returns invalid JSON, system retries up to 3 times
Exception Flow	If extraction fails, system shows error and prompts user to re-upload
Postconditions	Exam paper is ready for preview and download

6. Appendix

6.1 Revision History

Version	Date	Author	Changes
1.0	April 2025	Mujeeb ur Rehman & Madad Abbas	Initial SRS Document

6.2 Approval

Role	Name	Signature	Date
Student / Developer	Mujeeb ur Rehman (21886003)		
Student / Developer	Madad Abbas (21886010)		
FYP Supervisor	Syeda Asma bibi		
Department Head			

Document End — Software Requirements Specification v1.0